Establishing the Ad-Hoc Network Substrate

A functional and reliable ad-hoc network is the bedrock upon which the entire OLSR framework operates. Establishing such a network on commodity hardware like the Raspberry Pi is a non-trivial task that involves overcoming both hardware and software challenges. This section provides a detailed, step-by-step account of the process, which was structured into four distinct stages to ensure a robust and reproducible outcome. The process begins with the critical first step of driver installation for a compatible wireless adapter, moves to the precise network configuration required for IBSS mode, proceeds to the automation of this setup using a systemd service for reliability, and concludes with a final validation stage to confirm the substrate's performance and readiness.

Stage 1: Driver Installation and Kernel Module Configuration

The initial and most critical challenge was the selection of a suitable wireless adapter. A preliminary survey of commercially available USB Wi-Fi dongles revealed that most are designed for infrastructure mode and lack reliable support for IEEE 802.11 ad-hoc (IBSS) mode. The challenge was therefore twofold: identifying a dongle that explicitly supports IBSS mode and ensuring its drivers are stable on the Raspberry Pi OS platform.

The Realtek 8188FTV USB dongle was selected as it met both criteria. However, its driver is not included in the default Raspberry Pi OS distribution, necessitating manual installation. The driver was compiled from source and installed as a kernel module using the Dynamic Kernel Module Support (DKMS) framework, which ensures the driver is automatically rebuilt after kernel updates. The process, summarized in sequence of commands below, involves cloning the driver's source code, adding it to DKMS, and installing it.

```
# 1. Install dependencies and kernel headers
sudo apt update && sudo apt upgrade -y
sudo apt install -y build-essential git dkms raspberrypi-kernel-headers

# 2. Clone the driver source and install via DKMS
git clone https://github.com/kelebek333/rtl8188fu rtl8188fu-arm
cd rtl8188fu-arm
sudo dkms add .
sudo dkms install rtl8188fu/1.0

# 3. Copy firmware files
sudo mkdir -p /lib/firmware/rtlwifi/
sudo cp firmware/rtl8188fufw.bin /lib/firmware/rtlwifi/
```

Stage 2: Static IBSS Network Configuration

With the driver in place, the next step was to configure the wlan1 interface. To prevent the operating system's default network manager from interfering, the interface was first excluded

from the dhcpcd configuration by typing (`denyinterfaces wlan1`) in `dhcp.conf` file (`sudo nano /etc/dhcpcd.conf`). A script (`setup_adhoc_wlan1.sh`) was then created to execute the precise sequence of commands for establishing the IBSS network, as detailed in code sequence below. A key reliability enhancement was the use of a fixed BSSID in the `iw dev wlan1 ibss join` command, which forces all nodes to join a specific, identifiable ad-hoc cell, preventing network fragmentation.

```
#!/bin/bash
# Stop default network management services for wlan1
sudo systemctl stop wpa_supplicant@wlan1.service
sudo systemctl disable wpa_supplicant@wlan1.service

# Ensure a clean state before configuration
sudo ip link set wlan1 down
sudo ip addr flush dev wlan1

# Configure the interface for Ad-hoc (IBSS) mode
sudo iw dev wlan1 set type ibss
sudo ip link set wlan1 up

# Join a specific IBSS cell using a fixed BSSID for reliability
sudo iw dev wlan1 ibss join MyAdHocNetwork 2412 fixed-freq 02:11:22:33:44:55

# Assign a unique static IP address to the interface
sudo ip addr add 192.168.10.X/24 dev wlan1
```

Stage 3: Service Automation with `systemd`

To ensure the ad-hoc network was established automatically and reliably on every boot, the configuration script was wrapped into a `systemd` service. A service unit file, `adhoc-wlan1.service`, was created to manage the execution of the script. The definition of this service, shown in code sequence below, contains several important directives:

- `[Unit]` Section: The `After=network-online.target` directive ensures that this service runs only after the main networking stack is initialized. An additional `After=vncserver-x11-serviced.service` dependency was added to ensure that remote access services are available before the ad-hoc network is configured, which aids in debugging during startup.
- `[Service]` Section: `Type=oneshot` specifies that the script is expected to run once and `exit. RemainAfterExit=yes` tells `systemd` to consider the service "`active`" even after the script has finished, which is standard for one-time setup tasks.
- `[Install]` Section: `WantedBy=multi-user.target` ensures that the service is enabled as part of the standard multi-user boot process.

```
[Unit]
Description=Setup wlan1 for Ad-hoc network
After=network-online.target, vncserver-x11-serviced.service

[Service]
ExecStart=/usr/local/bin/setup_adhoc_wlan1.sh
Type=oneshot
RemainAfterExit=yes

[Install]
WantedBy=multi-user.target
```

After creating the service file, standard `systemctl` commands (`sudo systemctl daemon-reload sysyem` and `sudo systemctl enable adhoc-wlan1.service`) were used to reload the systemd daemon and enable the service, thus completing the automation process.

Stage 4: Configuration Verification

Once the automated systemd service was in place, a final verification stage was conducted to confirm that the wireless interfaces were configured exactly as intended. This step is crucial for ensuring the repeatability of the experiments. Standard Linux command-line tools were used for this purpose.

First, the operational mode of the network interfaces was inspected using the iwconfig command. As shown in the output in Figure 3.17, this check confirmed two critical facts:

1. The `wlan1` interface, connected to the external USB dongle, was correctly operating in "Ad-Hoc" mode.
2. The `wlan0` interface, the built-in chipset, remained in "Managed" mode, connected to the infrastructure network. This successfully validated the correct separation of the experimental and management network planes.

Next, the available data rates on the experimental interface were queried using `iwlist wlan1 rate`. The output, presented in Figure 3.18, confirmed that the interface had negotiated a set of basic IEEE 802.11b data rates, which was sufficient for the planned experiments.

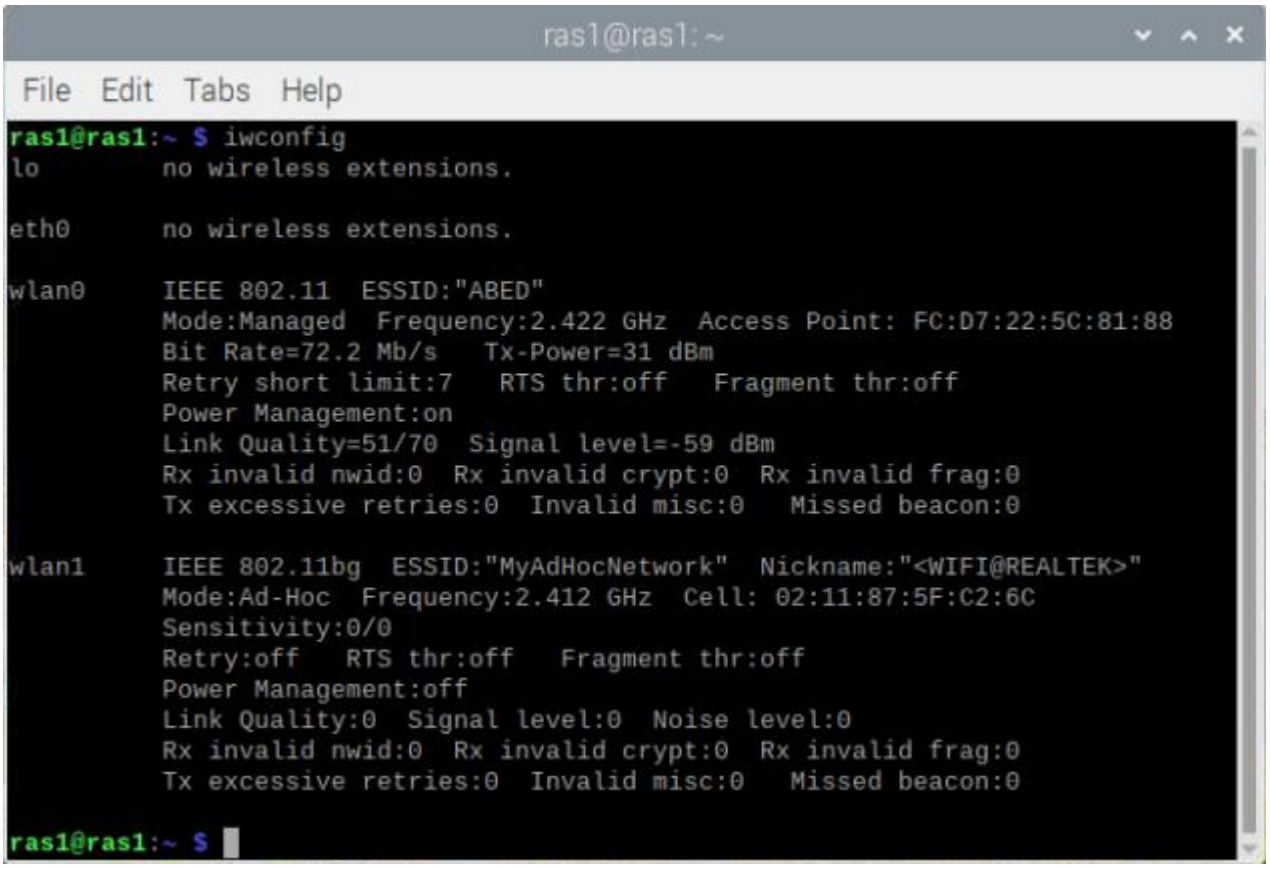

Figure خطأ! لا يوجد نص من النمط المعين في المستند.1 Verification of Interface Modes using iwconfig

Figure خطأ! لا يوجد نص من النمط المعين في المستند.2 Confirmation of Available Data Rates.

Stage 5: Substrate Performance and Viability Validation

The final stage in preparing the testbed was to move beyond configuration verification and conduct a rigorous validation of the wireless substrate's performance. It is critical to note that this entire validation phase was conducted before a single line of the OLSR framework was written. This methodical, bottom-up approach ensured that the underlying network could handle meaningful data exchange, thereby guaranteeing a functional and measurable foundation for the subsequent protocol implementation.

To achieve this, a comprehensive series of application-layer tests was performed over the established IBSS network. These experiments were designed to probe the substrate's behavior

and throughput under various conditions, utilizing both UDP and TCP protocols. Scenarios included one-to-one (unicast) transfers, one-to-many (broadcast) transmissions, and concurrent data flows.

For the purpose of brevity and relevance to the OLSR framework (which relies on UDP for its control messages) this section details one representative and foundational experiment: a large-scale UDP broadcast throughput test. This test was orchestrated using simple, custom-written Python scripts that leveraged the native socket library. A sender script was responsible for reading a 200 MB file and transmitting it as a stream of UDP broadcast datagrams, while a corresponding receiver script was run on the other four nodes.

The primary goal was to observe whether the network behaved as expected for a best-effort UDP broadcast over 802.11. The collective results are summarized in Figure 3.19. The figure documents that all receiving nodes successfully captured the vast majority of the data stream. As anticipated with inexpensive hardware (the USB dongles cost approximately $3 each) and an unacknowledged broadcast mechanism, minor packet loss and slight variations in throughput between nodes were observed. This behavior is consistent with the expected performance of such a setup.

Despite these expected variations, a functional end-to-end throughput, averaging approximately 5.3 Mbps, was consistently achieved across all receivers. This performance level, while not perfect, was deemed more than sufficient for the control message overhead and data loads anticipated for the OLSR protocol evaluation. The successful outcome of this experiment confirmed that the ad-hoc substrate, with all its real-world characteristics, provided a viable and predictable foundation for the main research.

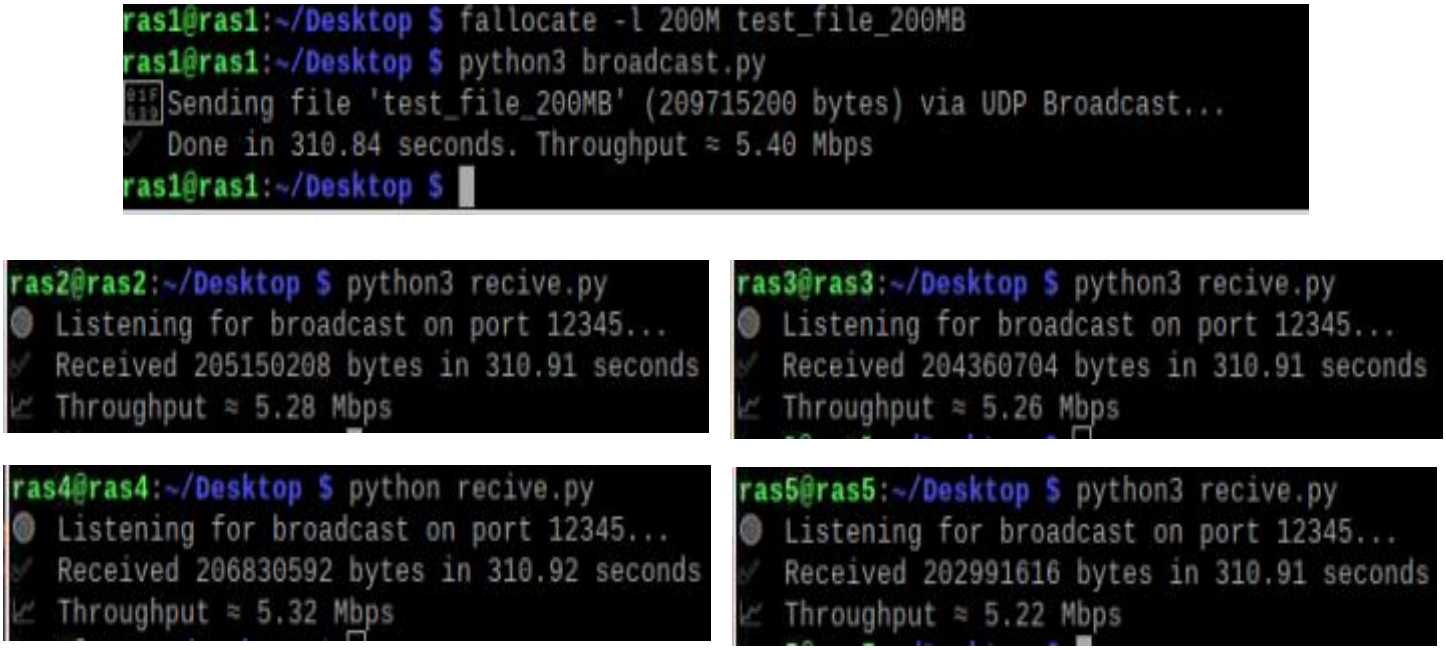

Figure 3 UDP Broadcast Throughput on the IBSS Substrate **خطأ! لا يوجد نص من النمط المعين في المستند.**